and so its TF will be high for many documents. Hence, this term 'player' may not help us to discriminate the set of documents that are more relevant to the user's query. However, the term 'wisden' is a more significant term as it is likely to be present only in the relevant documents. So we need to get those documents that contain the discriminating term 'wisden' and not include all those documents which contain the non-discriminating term 'player'.

This is where the inverse document frequency comes into play. Inverse document frequency (IDF) is an inverse of the measure of the number of documents that contain the term. Hence, for a discriminating term, since it would be present in fewer documents, its inverse document frequency would be much higher; hence, it can help offset the effect of non-discriminating terms for which the IDF will be much lower. Typically, both TF and IDF are scaled and normalised appropriately. Hence, IDF is typically defined as IDF (term t) = Log (N/Nt), where N is the total number of documents in the corpus and Nt is the total number of documents in which term 't' is present. I leave it to the reader to figure out whether IDF helps to improve precision, recall, or both.

Question (4) in last month's column was about predicting the results of election polls in states. This problem can be analysed in multiple different ways. Let's assume that we want to predict the winning party for a particular state, given that there are multiple parties contesting the election. We also know the past history of elections in that state, as well as various other features collected from past data. This can be modelled as a multi-label classification, where the winner is one of N parties contesting the elections.

On the other hand, if we are considering all the constituencies of the state, can we cluster the constituencies into different clusters such that each cluster represents the set of constituencies won by a specific party. We opt for clustering if we don't have labelled data for the winners in previous elections. In that case, we just want to use the features/attributes of different constituencies and cluster them. However, note that though we get a set of clusters, we don't know what these clusters represent. Hence, a human expert needs to label these clusters. Once these clusters are labelled, an unlabelled constituency can be labelled by assigning the cluster label that is closest to it. So this question can be modelled in multiple ways — either as a classification problem or as a clustering problem, depending on available data for analysis.

Question (8) was about word embeddings, which is the hottest topic currently in natural language processing. Frequency or count based approaches have earlier been widely used in natural language processing. For instance, latent semantic analysis or latent semantic indexing is one of the major approaches in this direction. These approaches have typically been based on global counts, wherein for each word, we consider all the words that have co-occurred with it, to construct its representation. Hence, the entire corpus has to be processed before we get a representation for each word based on co-occurrence counts.

On the other hand, word embeddings are based on a small local context window which is moved over all the contexts in the corpus and, at any point in time, a continuous vector representation is available for each word which gets updated as we examine more contexts. Note that co-occurrence based counts are typically sparse representations since a word will not co-occur with many other words in the vocabulary. On the other hand, embeddings are dense representations since they are of much lower dimensions.

A detailed discussion on the relation between embeddings and global count based representations can be found in the paper 'GloVe: Global Vectors for Word Representation' by Pennington et al available at *http://nlp.stanford.edu/pubs/glove.pdf*. It would be good for the readers to experiment with both Google word2vec embeddings available online (this includes word vectors for a vocabulary of three million words trained on a Google News dataset) as well as the Glove vectors available at *http://nlp.stanford.edu/projects/glove/*. Word embeddings and, subsequently, paragraph and document embeddings are widely used as inputs in various natural language processing tasks. In fact, they are used as input representation for many of the deep learning based text processing systems. There are a number of word2vec implementations available, the popular ones being from the Gensim package (*https://radimrehurek.com/gensim/models/word2vec.html)*, which is widely used. It would be useful for the readers to understand how word embeddings are learnt using neural networks without requiring any labelled data. While word embeddings are typically used in deep neural networks, the word2vec model for generating the word embeddings is not a deep architecture, but a shallow neural network architecture. More details can be found in the word2vec paper *https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf*.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, here's wishing all our readers a wonderful and productive year ahead! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist at Xerox India Research Centre. Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"gid=2339182*